

Single Step Backtesting Baselines

```
import pandas as pd
import numpy as np
import os, pathlib
from pathlib import Path

import joblib
import plotly.express as px
import plotly.graph_objects as go
import plotly.io as io

from functools import partial
```

```
import humanize
from sklearn.preprocessing import StandardScaler
from tqdm.autonotebook import tqdm
from IPython.display import display, HTML

# %load_ext autoreload
# %autoreload 2
np.random.seed(42)
import random

random.seed(42)
tqdm.pandas()
```

```
from google.colab import files
files.upload()
```

```
import gc
gc.collect()
'''
!unzip selected_blocks_test_feature_eng.parquet.zip
!unzip selected_blocks_train_feature_eng.parquet.zip
!unzip selected_blocks_val_feature_eng.parquet.zip
'''
!ls
```

```
!pip install statsforecast > /dev/null
```

```
import statsforecast
from statsforecast.core import StatsForecast
import utilsforecast
from utilsforecast.plotting import plot_series
from utilsforecast.losses import * #weighted_loss
from statsforecast.models import (
    Naive, SeasonalNaive, HoltWinters, ARIMA, Theta, TBATS, MSTL )
```

```
os.environ['NIXTLA_ID_AS_COL'] = '1'
```

```
def format_plot(fig, legends=None, xlabel="Time", ylabel="Value", title="", font_size=15):
    if legends:
        names = cycle(legends)
        fig.for_each_trace(lambda t: t.update(name=next(names)))

    fig.update_layout(
        autosize=False,
        width=900,
        height=500,
        title_text=title,
        title={
            "x": 0.5, "xanchor": "center", "yanchor": "top"},
        titlefont={"size": 20},
        legend_title=None,

        #legend=dict(font=dict(size=font_size), orientation="h", yanchor="bottom", y=0.98, xanchor="right"),

        yaxis=dict(
            title_text=ylabel,
            titlefont=dict(size=font_size),
            tickfont=dict(size=font_size),
        ),
        xaxis=dict(
            title_text=xlabel,
            titlefont=dict(size=font_size),
            tickfont=dict(size=font_size),
        ),
    )
    return fig
```

```
train_df = pd.read_parquet('selected_blocks_train_feature_eng.parquet')
```

```
test_df = pd.read_parquet('selected_blocks_test_feature_eng.parquet')
```

```
val_df = pd.read_parquet('selected_blocks_val_feature_eng.parquet')
```

```
gc.collect()
```

```
len(train_df), len(test_df), len(val_df)
```

```
(9120256, 409536, 470208)
```

```
len(train_df.LCLid.unique()), len(test_df.LCLid.unique()), len(val_df.LCLid.unique())
```

```
(317, 316, 316)
```

```
train_df.head(1)
```

	LCLid	timestamp	energy_consumption	timestamp_Week	frequency	energy_consumption_lag_1	energy_
0	MAC000026	2012-01-01	0.085	52	30min		NaN

1 rows x 78 columns

```
test_df.head(1)
```

	LCLid	timestamp	energy_consumption	timestamp_Week	frequency	energy_consumption_lag_1	ene
36576	MAC000026	2014-02-01	0.178	5	30min		0.202

1 rows x 78 columns

```
val_df.head(1)
```

	LCLid	timestamp	energy_consumption	timestamp_Week	frequency	energy_consumption_lag_1	ene
35088	MAC000026	2014-01-01	0.104	1	30min		0.12

1 rows x 78 columns

```
train_ = train_df[['LCLid', 'timestamp', 'energy_consumption']]
test_ = test_df[['LCLid', 'timestamp', 'energy_consumption']]
val_ = val_df[['LCLid', 'timestamp', 'energy_consumption']]
```

```
#del train_df, test_df, val_df
gc.collect()
```

```
''' create full training plus validation set for cross validation '''
train_val = pd.concat([train_, val_])
```

```
''' create full training plus validation set for cross validation '''
train_val_ts = pd.concat([train_, val_, test_])
```

```
train_val['LCLid'] = train_val['LCLid'].astype(str)
train_val_ts['LCLid'] = train_val_ts['LCLid'].astype(str)
```

```
train_val_ts.head()
```

	LCLid	timestamp	energy_consumption
0	MAC000026	2012-01-01 00:00:00	0.085
1	MAC000026	2012-01-01 00:30:00	0.108
2	MAC000026	2012-01-01 01:00:00	0.128
3	MAC000026	2012-01-01 01:30:00	0.159
4	MAC000026	2012-01-01 02:00:00	0.200

Naive Forecast

```
models = [Naive(), SeasonalNaive(season_length=48*7)]
model_names = [model.__class__.__name__ for model in models]
```

```
sf = StatsForecast(
    models=models, freq='30min', n_jobs=-1)
```

```
cross_validation_val_df = sf.cross_validation(
    df = train_val,
    h = 1,
    step_size = 1,
    n_windows = len(val_.timestamp.unique()),
    id_col = 'LCLid',
    time_col = 'timestamp',
    target_col = 'energy_consumption'
)
```

```
cross_validation_val_df.head()
```

	LCLid	timestamp	cutoff	energy_consumption	Naive	SeasonalNaive
0	MAC000026	2014-01-01 00:00:00	2013-12-31 23:30:00	0.104	0.120	0.180
1	MAC000026	2014-01-01 00:30:00	2014-01-01 00:00:00	0.157	0.104	0.139
2	MAC000026	2014-01-01 01:00:00	2014-01-01 00:30:00	0.111	0.157	0.098
3	MAC000026	2014-01-01 01:30:00	2014-01-01 01:00:00	0.116	0.111	0.147
4	MAC000026	2014-01-01 02:00:00	2014-01-01 01:30:00	0.152	0.116	0.137

```
cross_validation_test_df = sf.cross_validation(
    df = train_val_ts,
    h = 1,
    step_size = 1,
    n_windows = len(test_.timestamp.unique()),
    id_col = 'LCLid',
    time_col = 'timestamp',
    target_col = 'energy_consumption'
)
```

```
cross_validation_test_df.head()
```

	LCLid	timestamp	cutoff	energy_consumption	Naive	SeasonalNaive
0	MAC000026	2014-02-01 00:00:00	2014-01-31 23:30:00	0.178	0.202	0.124
1	MAC000026	2014-02-01 00:30:00	2014-02-01 00:00:00	0.092	0.178	0.109
2	MAC000026	2014-02-01 01:00:00	2014-02-01 00:30:00	0.115	0.092	0.110
3	MAC000026	2014-02-01 01:30:00	2014-02-01 01:00:00	0.135	0.115	0.124
4	MAC000026	2014-02-01 02:00:00	2014-02-01 01:30:00	0.143	0.135	0.146

```
import utilsforecast
from utilsforecast.evaluation import evaluate
```

```
def forecast_bias_NIXTLA(y, y_hat):
    return np.mean(y - y_hat)

forecast_mase = partial(mase, seasonality=1)
forecast_mase.__name__ = "mase"
forecast_bias_NIXTLA.__name__ = "forecast_bias"
```

```
def forecast_bias(df, models, ids=None, **kwargs):
    # Calculate bias per unique ID as required by utilsforecast.evaluation.evaluate
    res = df.groupby('LCLid').apply(
        lambda x: pd.Series({model: (x['y'] - x[model]).mean() for model in models})
    ).reset_index()
    return res

forecast_bias.__name__ = 'forecast_bias'

# Re-run evaluation for validation set
baseline_val_metrics_df = evaluate(
    df = cross_validation_val_df.drop(['cutoff'], axis=1).rename(columns={'energy_consumption': 'y'}),
    metrics = [mse, mae, rmse, forecast_mase, forecast_bias],
    models = model_names,
    train_df = train_val[['timestamp', 'LCLid', 'energy_consumption']].rename(columns={'energy_consumpt
id_col = 'LCLid',
time_col = 'timestamp',
target_col = 'y'
)
```

DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future ve

baseline_val_metrics_df.head(2)

	LCLid	metric	Naive	SeasonalNaive
0	MAC000026	mse	0.016588	0.024459
1	MAC000052	mse	0.037834	0.100394

baseline_val_metrics_df[(baseline_val_metrics_df.metric == 'forecast_bias') & (baseline_val_metrics_df.

	LCLid	metric	Naive	SeasonalNaive
1583	MAC005244	forecast_bias	0.000003	-0.017269

baseline_val_metrics_df_pivot = (baseline_val_metrics_df.melt(id_vars = ['LCLid','metric'], value_vars
baseline_val_metrics_df_pivot.head(10)

	metric	LCLid	Algorithm	forecast_bias	mae	mase	mse	rmse
0		MAC000026	Naive	0.000055	0.057191	0.677547	0.016588	0.128794
1		MAC000026	SeasonalNaive	0.015694	0.065042	0.770564	0.024459	0.156392
2		MAC000052	Naive	-0.000656	0.094665	1.554053	0.037834	0.194509
3		MAC000052	SeasonalNaive	-0.012365	0.168205	2.761302	0.100394	0.316850
4		MAC000079	Naive	-0.000177	0.383802	2.806964	0.267319	0.517029
5		MAC000079	SeasonalNaive	0.008796	0.544846	3.984767	0.594990	0.771356
6		MAC000085	Naive	0.000212	0.168372	0.892778	0.084855	0.291298
7		MAC000085	SeasonalNaive	0.029804	0.391136	2.073964	0.408503	0.639143
8		MAC000105	Naive	-0.000054	0.247136	1.025511	0.175365	0.418766
9		MAC000105	SeasonalNaive	0.102055	0.605969	2.514513	0.829056	0.910525

```
# Re-run evaluation for test set to include the fixed forecast_bias per ID
baseline_test_metrics_df = evaluate(
    df = cross_validation_test_df.drop(['cutoff'], axis=1).rename(columns={'energy_consumption': 'y'}),
    metrics = [mse, mae, rmse, forecast_mase, forecast_bias],
    models = model_names,
    train_df = train_val[['timestamp', 'LCLid', 'energy_consumption']].rename(columns={'energy_consumpt
id_col = 'LCLid',
time_col = 'timestamp',
target_col = 'y'
)
```

/tmp/ipykernel_107843/2701502557.py:3: DeprecationWarning:

DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future ve

gc.collect()

82

baseline_test_metrics_df.head()

	LCLid	metric	Naive	SeasonalNaive
0	MAC000026	mse	0.050425	0.123446
1	MAC000052	mse	0.037619	0.113184
2	MAC000079	mse	0.239782	0.402946
3	MAC000085	mse	0.066247	0.166456
4	MAC000105	mse	0.000572	1.024993

```
# Re-pivot the test metrics to include forecast_bias
baseline_test_metrics_df_pivot = (baseline_test_metrics_df.melt(id_vars = ['LCLid','metric'], value_var
baseline_test_metrics_df_pivot.head(10)
```

metric	LCLid	Algorithm	forecast_bias	mae	mase	mse	rmse
0	MAC000026	Naive	0.000242	0.128323	1.520261	0.050425	0.224555
1	MAC000026	SeasonalNaive	0.024704	0.225677	2.673618	0.123446	0.351349
2	MAC000052	Naive	-0.000031	0.098455	1.616270	0.037619	0.193957
3	MAC000052	SeasonalNaive	-0.022894	0.185111	3.038838	0.113184	0.336428
4	MAC000079	Naive	0.000211	0.366995	2.684039	0.239782	0.489675
5	MAC000079	SeasonalNaive	-0.011179	0.444154	3.248352	0.402946	0.634780
6	MAC000085	Naive	0.000252	0.134926	0.715432	0.066247	0.257385
7	MAC000085	SeasonalNaive	-0.056519	0.258485	1.370594	0.166456	0.407990
8	MAC000105	Naive	0.000036	0.019388	0.080453	0.000572	0.023918
9	MAC000105	SeasonalNaive	-0.525917	0.537992	2.232439	1.024993	1.012420

```
naive_pred_val_df = (cross_validation_val_df
    .rename(columns = {'y':'energy_consumption','Naive':'naive_predictions'})
    .drop(['cutoff','SeasonalNaive'],axis=1).set_index('timestamp'))

naive_pred_test_df = (cross_validation_test_df
    .rename(columns = {'y':'energy_consumption','Naive':'naive_predictions'})
    .drop(['cutoff','SeasonalNaive'],axis=1).set_index('timestamp'))
```

```
seasonal_naive_pred_val_df = (cross_validation_val_df
    .rename(columns = {'y':'energy_consumption','SeasonalNaive':'naive_predictions'})
    .drop(['cutoff','Naive'],axis=1).set_index('timestamp'))

seasonal_naive_pred_test_df = (cross_validation_test_df
    .rename(columns = {'y':'energy_consumption','SeasonalNaive':'naive_predictions'})
    .drop(['cutoff','Naive'],axis=1).set_index('timestamp'))
```

```
baseline_val_metrics_df = baseline_val_metrics_df_pivot.copy()
baseline_test_metrics_df = baseline_test_metrics_df_pivot.copy()
```

Overall Metrics

```
def forecast_bias_aggregate(y, y_hat):
    return np.mean(y - y_hat) * 100

overall_metrics_naive_val = {
    "MAE": mae(cross_validation_val_df, models=["Naive"],
        target_col="energy_consumption", id_col="LCLid")["Naive"].mean(),
    "MSE": mse(cross_validation_val_df, models=["Naive"],
        target_col="energy_consumption", id_col="LCLid")["Naive"].mean(),
    "meanMASE": baseline_val_metrics_df[baseline_val_metrics_df.Algorithm ==
        'Naive']["mase"].mean(),
    "Forecast Bias": forecast_bias_aggregate(cross_validation_val_df
        ["energy_consumption"], cross_validation_val_df["Naive"])
}

overall_metrics_naive_val

{'MAE': np.float64(0.10537660330117209),
'MSE': np.float64(0.0521153665942322),
'meanMASE': np.float64(1.110505963963964),
'Forecast Bias': np.float64(0.0038386221412235483)}
```

```
overall_metrics_snaive_val = {
    "MAE": mae(cross_validation_val_df, models=["SeasonalNaive"], target_col="energy_consumption", id_c
    "MSE": mse(cross_validation_val_df, models=["SeasonalNaive"], target_col="energy_consumption", id_c
    "meanMASE": baseline_val_metrics_df[baseline_val_metrics_df.Algorithm == 'SeasonalNaive']["mase"].n
    "Forecast Bias": forecast_bias_aggregate(cross_validation_val_df["energy_consumption"], cross_valic
}

overall_metrics_snaive_val

{'MAE': np.float64(0.17318038391945215),
'MSE': np.float64(0.11989362026109299),
'meanMASE': np.float64(1.8536930416030868),
'Forecast Bias': np.float64(0.1495446963014547)}
```

```
overall_metrics_naive_test = {
    "MAE": mae(cross_validation_test_df, models=["Naive"], target_col="energy_consumption", id_col="LCI"),
    "MSE": mse(cross_validation_test_df, models=["Naive"], target_col="energy_consumption", id_col="LCI"),
    "meanMASE": baseline_test_metrics_df[baseline_test_metrics_df.Algorithm == 'Naive']["mase"].mean(),
    "Forecast Bias": forecast_bias_aggregate(cross_validation_test_df["energy_consumption"], cross_vali
}

overall_metrics_naive_test
```

```
{'MAE': np.float64(0.10387007169911233),
'MSE': np.float64(0.05142791197638054),
'meanMASE': np.float64(1.10749320647332),
'Forecast Bias': np.float64(-6.660188450579126e-05)}
```

```
overall_metrics_snaive_test = {
    "MAE": mae(cross_validation_test_df, models=["SeasonalNaive"], target_col="energy_consumption", id_
    "MSE": mse(cross_validation_test_df, models=["SeasonalNaive"], target_col="energy_consumption", id_
    "meanMASE": baseline_test_metrics_df[baseline_test_metrics_df.Algorithm == 'SeasonalNaive']["mase"]
    "Forecast Bias": forecast_bias_aggregate(cross_validation_test_df["energy_consumption"], cross_vali
}

overall_metrics_snaive_test
```

```
{'MAE': np.float64(0.16736701735324466),
'MSE': np.float64(0.11045058202350522),
'meanMASE': np.float64(1.8108778206577365),
'Forecast Bias': np.float64(-0.7373658152221639)}
```

Evaluate Baseline Forecast

```
agg_metric_val_df = pd.DataFrame([overall_metrics_naive_val, overall_metrics_snaive_val], index=["Naive", "Seasonal Naive"])

agg_metric_val_df.style.format({"MAE": "{:.3f}",
                                "MSE": "{:.3f}",
                                "meanMASE": "{:.3f}",
                                "Forecast Bias": "{:.2f}%"}).highlight_min(color='lightgreen')
```

	MAE	MSE	meanMASE	Forecast Bias
Naive	0.105	0.052	1.111	0.00%
Seasonal Naive	0.173	0.120	1.854	0.15%

```
agg_metric_test_df = pd.DataFrame([overall_metrics_naive_test, overall_metrics_snaive_test], index=["Naive", "Seasonal Naive"])

agg_metric_test_df.style.format({"MAE": "{:.3f}",
                                "MSE": "{:.3f}",
                                "meanMASE": "{:.3f}",
                                "Forecast Bias": "{:.2f}%"}).highlight_min(color='lightgreen')
```

	MAE	MSE	meanMASE	Forecast Bias
Naive	0.104	0.051	1.107	-0.00%
Seasonal Naive	0.167	0.110	1.811	-0.74%

baseline_val_metrics_df_pivot

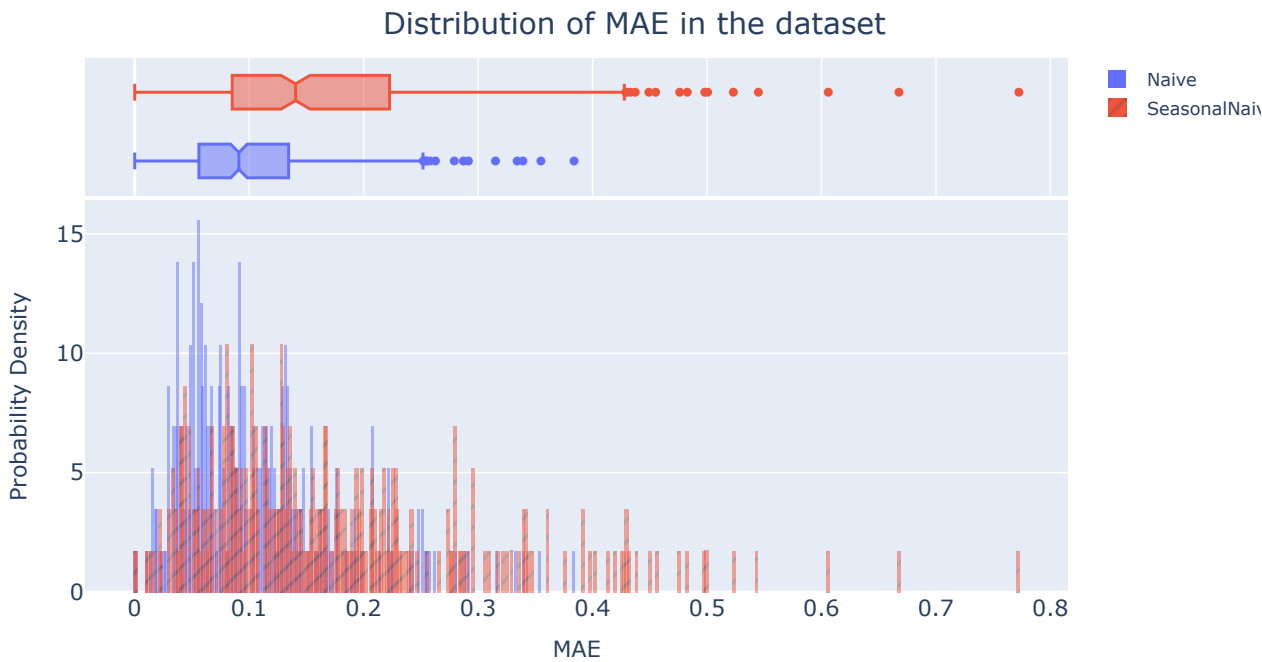
metric	LCLid	Algorithm	forecast_bias	mae	mase	mse	rmse
0	MAC000026	Naive	0.000055	0.057191	0.677547	0.016588	0.128794
1	MAC000026	SeasonalNaive	0.015694	0.065042	0.770564	0.024459	0.156392
2	MAC000052	Naive	-0.000656	0.094665	1.554053	0.037834	0.194509
3	MAC000052	SeasonalNaive	-0.012365	0.168205	2.761302	0.100394	0.316850
4	MAC000079	Naive	-0.000177	0.383802	2.806964	0.267319	0.517029
...	...	...	...	...	...	...	...
573	MAC005224	SeasonalNaive	-0.011247	0.163475	1.305122	0.096028	0.309883
574	MAC005244	Naive	0.000003	0.147940	1.147496	0.064772	0.254504
575	MAC005244	SeasonalNaive	-0.017269	0.277968	2.156063	0.177819	0.421686
576	MAC005249	Naive	0.000202	0.170077	1.126861	0.123579	0.351539
577	MAC005249	SeasonalNaive	0.000968	0.133976	0.887672	0.090650	0.301082

578 rows x 7 columns

```
fig = px.histogram(baseline_val_metrics_df_pivot,
                    x="mae",
                    color="Algorithm",
                    pattern_shape="Algorithm",
                    marginal="box",
                    nbins=500,
                    barmode="overlay",
                    histnorm="probability density")

fig = format_plot(fig, xlabel="MAE", ylabel="Probability Density", title="Distribution of MAE in the dataset")

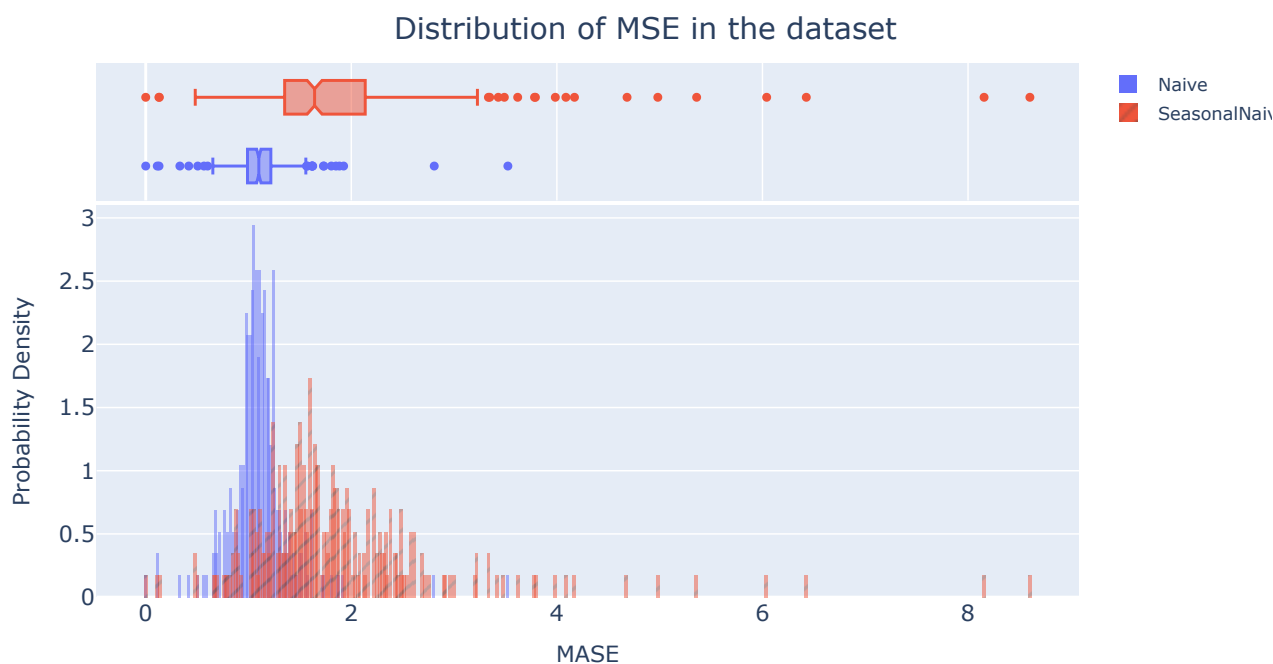
#fig.update_layout(xaxis_range=[0,3.2])
fig.show()
```



```
fig = px.histogram(baseline_val_metrics_df_pivot,
                    x="mase",
                    color="Algorithm",
                    pattern_shape="Algorithm",
                    marginal="box",
                    nbins=500,
                    barmode="overlay",
                    histnorm="probability density")

fig = format_plot(fig, xlabel="MASE", ylabel="Probability Density", title="Distribution of MASE in the dataset")

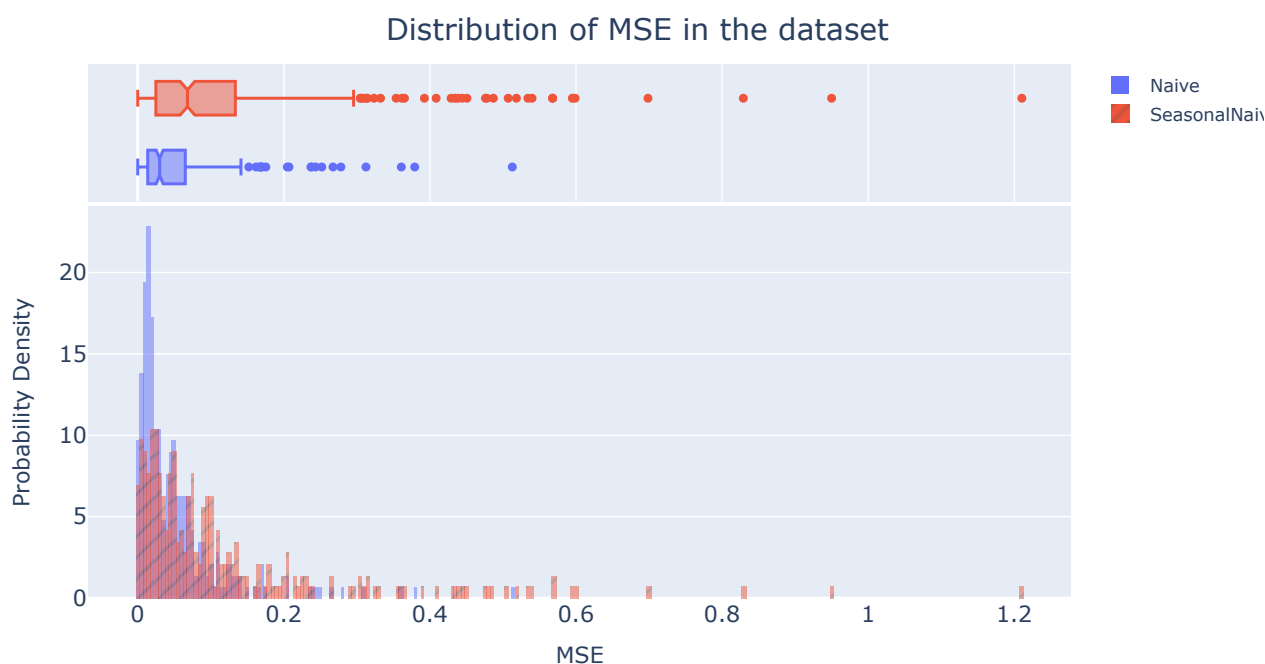
#fig.update_layout(xaxis_range=[0,3.2])
fig.show()
```



```
fig = px.histogram(baseline_val_metrics_df_pivot,
                   x="mse",
                   color="Algorithm",
                   pattern_shape="Algorithm",
                   marginal="box",
                   nbins=500,
                   barmode="overlay",
                   histnorm="probability density")

fig = format_plot(fig, xlabel="MSE", ylabel="Probability Density", title="Distribution of MSE in the dataset")

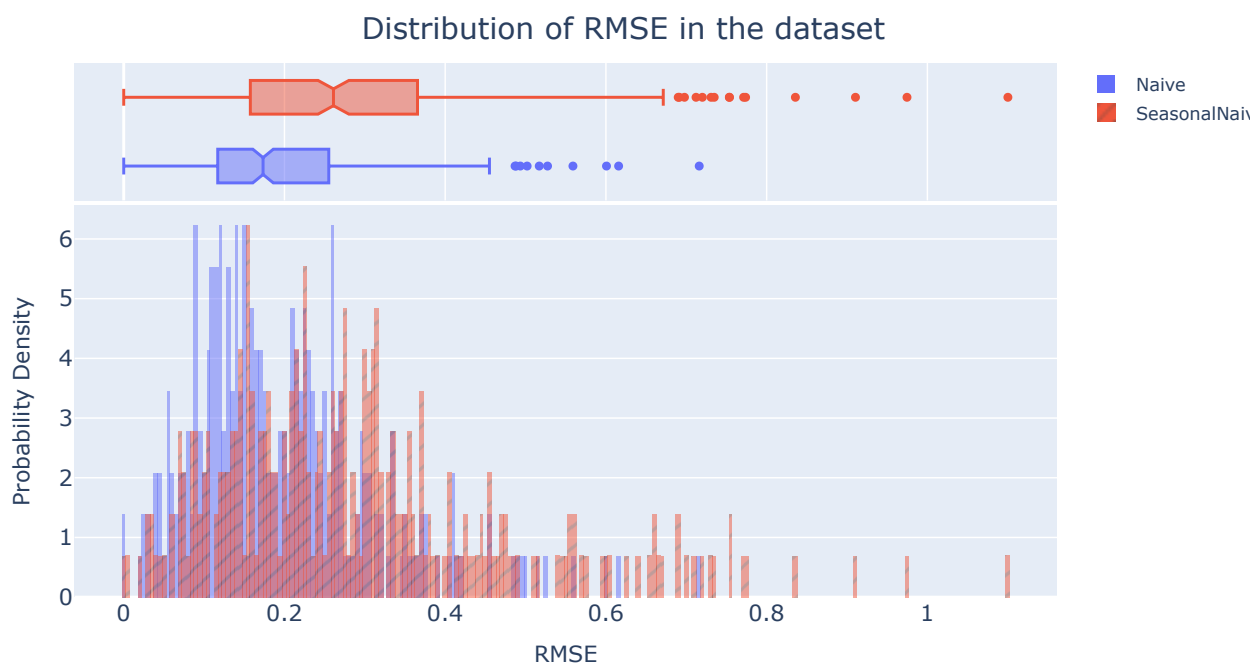
#fig.update_layout(xaxis_range=[0,3.2])
fig.show()
```



```
fig = px.histogram(baseline_val_metrics_df_pivot,
                   x="rmse",
                   color="Algorithm",
                   pattern_shape="Algorithm",
                   marginal="box",
                   nbins=500,
                   barmode="overlay",
                   histnorm="probability density")

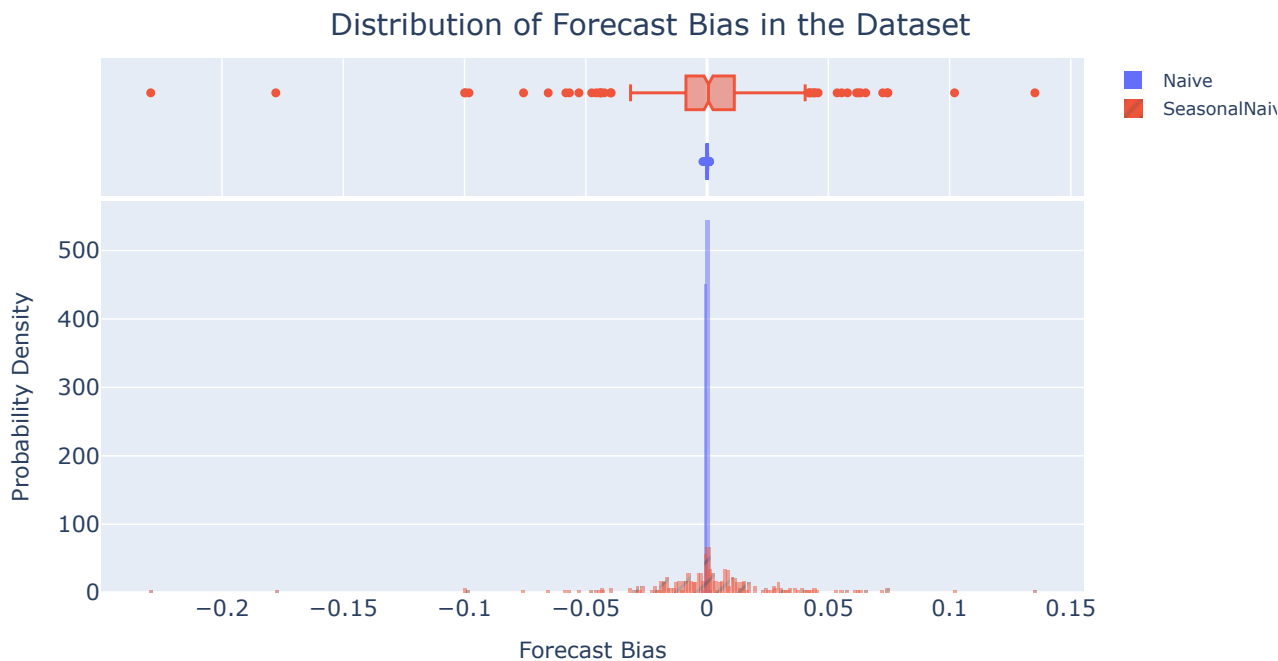
fig = format_plot(fig, xlabel="RMSE", ylabel="Probability Density", title="Distribution of RMSE in the dataset")

#fig.update_layout(xaxis_range=[0,3.2])
fig.show()
```

```
fig = px.histogram(baseline_val_metrics_df_pivot,
x="forecast_bias",
color="Algorithm",
pattern_shape="Algorithm",
marginal="box",
nbins=500,
barmode="overlay",
histnorm="probability density")

fig = format_plot(fig, xlabel="Forecast Bias", ylabel="Probability Density", title="Distribution of For
fig.show()
```



```
baseline_pred_val_df = naive_pred_val_df.reset_index().merge(seasonal_naive_pred_val_df.reset_index().c
baseline_pred_test_df = naive_pred_test_df.reset_index().merge(seasonal_naive_pred_test_df.reset_index(
```

```
baseline_pred_val_df.head()
```

	timestamp	LCLid	energy_consumption	naive_predictions_x	naive_predictions_y
0	2012-04-08 08:00:00	MAC005249	0.218	0.099	0.159
1	2012-04-08 08:30:00	MAC005249	1.098	0.218	0.139
2	2012-04-08 09:00:00	MAC005249	0.126	1.098	0.230
3	2012-04-08 09:30:00	MAC005249	0.190	0.126	0.187
4	2012-04-08 10:00:00	MAC005249	0.191	0.190	1.564

baseline_pred_test_df.head()

	timestamp	LCLid	energy_consumption	naive_predictions_x	naive_predictions_y
0	2012-04-12 08:00:00	MAC005249	0.187	0.137	0.130
1	2012-04-12 08:30:00	MAC005249	0.104	0.187	0.195
2	2012-04-12 09:00:00	MAC005249	0.188	0.104	0.129
3	2012-04-12 09:30:00	MAC005249	0.129	0.188	0.215
4	2012-04-12 10:00:00	MAC005249	0.153	0.129	0.150

baseline_pred_val_df.to_pickle(
 'single_step_backtesting_baseline_pred_val_df.pkl')